UNIVERSITY
OF WOLLONGONG
IN DUBAI

CSCI323 Project Report Report

Name	Student ID
Mohammed Atwani	8485811
Sayed Elyas	8479574
Saad Bin Waqas	8186388

UNIVERSITY
OF WOLLONGONG
IN DUBAI

Github Repository Link

https://github.com/AtwaniGG/CSCI323-AI-Based-Property-Price-Prediction-in-Dubai

Executive Summary

This project is a solution proposed for the artificial intelligence applications in the prediction of the price of residential property based on square meters (AED/sqm) in the real estate market in Dubai. Using a dataset of more than 1.6 million historical transactions from the Dubai Land Department, the system applies distributed big data processing using Apache Spark and deep learning using PyTorch to provide accurate pricing forecasting.

Four different modeling segments have been implemented and compared, namely; a baseline Decision Tree, a Bagging ensemble, a Gradient Boosting ensemble (all implemented from scratch using Spark ML), and a Multi-Layer Perceptron (MLP) Neural Network using PyTorch. After a lot of feature engineering work (106 predictive features) and evaluation, the Bagging Ensemble model achieved the least amount of RMSE among Spark-based models at 3,273.04 AED/sqm while the Neural Network-MLP achieved an R2 score of 0.7625 with an RMSE of 3,810.88 AED/sqm and MAE of 2,735.13 AED/sqm.

These results prove that AI-based property valuation can be a good source of pricing instructions for investors, developers, and regulators in one of the most dynamic property markets in the world.

UNIVERSITY
OF WOLLONGONG
IN DUBAI

Table of Contents

Github Repository Link **2**
Executive Summary **2**
1. Introduction and Motivation **4**
1.1 Industry Background 4
1.2 Problem Statement 4
1.3 Why AI Is The Appropriate Solution 4
1.4 Business Objectives and Success Metrics 5
2. Literature Review **5**
3. Technical Requirements and Analysis **6**
3.1 System Requirements 6
3.2 Data Requirements 6
3.3 Scalability Considerations 6
4. Solution Methodology and Implementation **6**
4.1 AI Techniques Selected 6
4.2 Data Preprocessing 7
4.3 Feature Engineering 7
4.3.1 Table 1: Feature Categories 7
4.4 Algorithm Implementation 8
4.5 Training and Validation 9
5. Results and Evaluations **9**
5.1 Spark Model Performance (10-Fold CV) 9
5.2 Neural Network MLP Performance 9
5.3 Final Model Comparison 10
5.4 Business Impact 10
6. Limitations, Challenges, and Constraints **11**
7. Conclusions and Discussion **11**
7.1 Summary of Findings 11
7.2 Achievement of Objectives 12
7.3 Key Lessons Learned 12
8. Recommendations and Future Work **12**
9. References **13**
10. Appendix **14**

1. Introduction and Motivation

1.1 Industry Background

Dubai's real estate market is one of the busiest in the world. The Dubai Land Department has registered more than 1.6 million property transactions in the residential, commercial, and industrial segments. The market is marked by high volatility, fast development cycles and large fluctuations in prices from neighborhood to neighborhood and property type to property type. Property valuation in Dubai has traditionally been based on manual appraisals and comparative market analyses. These methods are labor intensive and subjective and are often behind market conditions. The number of transactions is so high that manual analysis at scale is not possible.

1.2 Problem Statement

Accurately predicting property prices per square meter in Dubai is a complex challenge in light of the interplay of many factors: location, property type, bedrooms, proximity of landmarks and metro stations, type of transaction, and temporal market trends. Simple statistical models cannot account for the non-linear relationships in the pricing of real estate.
The central question: Is it possible with these AI models, which are trained on historical transaction data, to predict with a high enough accuracy, which is sufficient to provide automatic valuation and market analysis of residential property per square meter?

1.3 Why AI Is The Appropriate Solution

The problem is well-suited for AI due to:

1. **High dimensionality:** Prices depend on dozens of interacting features, making manual modeling impractical.
2. **Non-linear relationships:** A penthouse in a prime area behaves differently from a studio in the same location.
3. **Large-scale data:** 1.6 million transactions supports training complex models with distributed processing.
4. **Pattern recognition:** AI can discover latent patterns that human analysts may overlook.

1.4 Business Objectives and Success Metrics

1. Build an end-to-end pipeline from raw data to trained models.

2. Compare traditional ensemble methods with deep learning for tabular real estate data.
3. Achieve an R^2 score above 0.70.
4. Minimize RMSE and MAE for practically useful predictions.
5. Ensure reproducibility through Docker containerization.

2. Literature Review

Hedonic pricing models were proposed by Rosen (1974) to form the basis for property valuation by breaking prices down into constituent characteristics. However, these linear models have problems with non-linear interactions. Limsombunchai (2004) showed that Artificial Neural Networks are more superior to the hedonic model in predicting housing prices when there are non-linear relationships.

Gradient Boosting was proposed by Friedman (2001) where weak learners are built sequentially, and each tree corrects residual errors of earlier trees. This method has been very successful for structured data (Chen & Guestrin, 2016). While there are optimized implementations from libraries such as XGBoost, LightGBM, implementing the algorithm from scratch gives a better understanding of the algorithm.

For neural networks on tabular data, Kadra et al. (2021) demonstrated that well-regularized MLPs can be on the same level as tree-based methods with the use of suitable techniques like dropout and correct scaling.

Apache Spark is now a standard in the field of big data processing and machine learning (Zaharia et al., 2016). Spark MLlib offers distributed implementations of algorithms and PySpark allows for workflows in Python at scale.

Selim (2009) and Kok et al. (2014) discussed models for the real estate market in the Middle East, where location, size and amenity proximity turned out to be major price determinants. Our work goes beyond this, applying modern AI techniques to the comprehensive Dubai Land Department dataset, and at a scale that has not been previously addressed.

3. Technical Requirements and Analysis

3.1 System Requirements

Requirement	Specification
Processing Framework	Apache Spark 3.5.0
Deep Learning Framework	Pytorch
Runtime Environment	Docker
Python Version	Python 3.11

3.2 Data Requirements

The data to be used in the project is the Dubai Land Department Transactions dataset with 1,644,775 records in 46 columns (279 MB CSV). Features are property type, sub-type, area, bedrooms, parking, location, near landmarks/metro/malls, transaction type and date. The target variable is actual_worth but transformed to price per square meter.

3.3 Scalability Considerations

- Docker containerization ensures consistent execution across platforms.
- Apache Spark distributes computation and can scale to cluster deployments.
- Modular pipeline: Preprocessing and modeling are decoupled for independent updates.
- The Checkpoint mechanism handles large joins without memory overflow.

4. Solution Methodology and Implementation

4.1 AI Techniques Selected

Four modeling approaches were selected:

1. Baseline Decision Tree (Spark ML): Reference point for ensemble improvements. Handles mixed data types natively.
2. Bagging Ensemble (from scratch, Spark): Trains multiple trees on random data subsets and averages predictions, reducing variance.
3. Gradient Boosting (from scratch, Spark): Trains trees sequentially on residual errors, reducing bias. Configuration: 10 estimators, learning rate 0.1, max depth 5.
4. Neural Network MLP (PyTorch): Three hidden layers to capture complex non-linear feature interactions.

4.2 Data Preprocessing

The raw dataset undergoes the following transformations using PySpark:

1. Bilingual cleanup: Arabic columns (suffix _ar) dropped; English suffixes (_en) removed.

2. ID column removal: Redundant IDs dropped in favor of human-readable names.
3. Procedure grouping: Raw procedure names simplified into Sales, Mortgage, Grants, Lease-to-Own, and Other.
4. Date parsing: instance_date parsed into sale_year and sale_quarter.
5. Bedroom extraction: Heterogeneous rooms values ("STUDIO", "3 B/R", "PENTHOUSE") parsed into numeric bedrooms and binary is_penthouse.
6. Property usage encoding: Decomposed into binary flags (residential, commercial, industrial, etc.).
7. Null handling: Defaults applied — master_project → "Standalone", nearest_landmark → "No Landmark", nearest_metro → "No Metro Access", nearest_mall → "No Major Mall".
8. Log transformation: Target variable log-transformed to reduce skewness.

4.3 Feature Engineering

The cleaned dataset is further processed for modeling:

1. Residential filtering: Non-residential transactions removed (94,435 → 80,791 rows, 14.4% removed).
2. Target derivation: Log-transformed value exponentiated back, divided by area to get price_per_sqm. Outliers removed using 1st–99th percentile, yielding 79,127 rows (range: 502–44,367 AED/sqm, mean: 13,003).
3. Market-history features: Area-level aggregates per year/quarter — mean price, median price, transaction count, and momentum.
4. Target encoding: High-cardinality features (area_name, master_project) encoded using mean target encoding with global mean imputation.
5. One-hot encoding: Low-cardinality categoricals (property_type, property_sub_type, reg_type, nearest_landmark, nearest_metro, nearest_mall, procedure_group) encoded via Spark ML Pipeline.
6. Feature assembly: All features combined into a 106-dimensional vector using VectorAssembler.

4.3.1 Table 1: Feature Categories

Category	Count	Example
Numerical Base	6	procedure_area, bedrooms, has_parking, is_penthouse, sale_year, sale_quarter
Market History	4	area_mean_price,

		area_median_price, area_transaction_count, area_momentum
Target Encoded	2	area_name_encoded, master_project_encoded
One Hot Encoded	7 cols (~94 dims)	property_type, procedure_group, nearest_metro, etc.
Total	106	

4.4 Algorithm Implementation

Gradient Boosting (From Scratch):

1. Initialize all predictions to zero.
2. For each of 10 iterations: compute residuals, train a DecisionTreeRegressor (max depth 5) on residuals, update predictions with learning_rate × tree_output (learning rate = 0.1).
3. Final prediction is the sum of all scaled tree outputs.

Neural Network MLP (PyTorch):
Input (106 features) → Linear(106, 128) + ReLU + Dropout(0.3) → Linear(128, 64) + ReLU → Linear(64, 32) + ReLU → Linear(32, 1) Output

- Dropout (0.3) on the first hidden layer prevents overfitting.
- StandardScaler on features and target for training stability.
- Adam optimizer (lr=0.001) adapts learning rates per parameter.
- MSELoss directly optimizes for the regression objective.
- Batch size 32, 50 epochs with early stopping (patience=10).

4.5 Training and Validation

Spark models: 10-fold cross-validation with deterministic hash-based fold assignment. Spark RDD checkpointing manages memory during iterative training.

Neural Network: 80/20 train/validation split (seed=42). Feature and target scaling with inverse transformation for evaluation. Automatic GPU detection with CPU fallback.

5. Results and Evaluations

5.1 Spark Model Performance (10-Fold CV)

Table 2: 10-Fold Cross-Validation Results

Model	Avg RMSE (AED/sqm)	Std RMSE	Avg MAE (AED/sqm)	Std MAE
Baseline (Single Tree)	3,710.28	96.93	2,363.60	44.21
Bagging Ensemble	3,273.04	69.23	2,139.71	30.78
Gradient Boosting	5,732.31	66.81	4,,395.89	45.5

Key observations:

- **Bagging** achieved the best RMSE at 3,273.04 AED/sqm — an 11.78% improvement over the baseline.
- **Gradient Boosting** had the highest RMSE but the lowest standard deviation (66.81), indicating the most consistent predictions across folds. The higher error suggests the conservative learning rate and limited estimators led to underfitting.
- **Variance** decreased with ensemble complexity: baseline Std RMSE 96.93 → Bagging 69.23 → Gradient Boosting 66.81.

5.2 Neural Network MLP Performance

Table 3: Neural Network Evaluation (Train/Val Split)

Metric	Value
RMSE	3,,810.88 AED/sqm
MAE	2,,735.13 AED/sqm
R^2 Score	0.7625

The Neural Network explains 76.25% of the variance in property prices, exceeding the project target of 0.70. Given the mean price of 13,003 AED/sqm, the MAE represents approximately 21% mean absolute percentage error.

5.3 Final Model Comparison

Table 4: Final Model Comparison

Model	RMSE	MAE	R^2	Evaluation	Framework
Baseline Decision Tree	3,,710.28	2,363.60	0.62	10-Fold CV	Spark ML
Bagging Ensemble	3,273.04	2,139.71	0.59	10-Fold CV	Spark ML
Random Forest	5,732.31	0.4042	0.6721	10-Fold CV	Spark ML
Neural Networks	3,810.88	2,735.13	0.7625	Train/Val	Pytorch

5.4 Business Impact

- The Bagging Ensemble offers the best raw accuracy for Spark-deployed scenarios.
- The Neural Network provides the strongest explanatory power through its R^2 score.
- For a typical 100 sqm apartment valued at ~1,300,000 AED, the MAE translates to approximately 273,500 AED deviation — useful for ranking and market analysis, though point estimates require further refinement for formal appraisals.

6. Limitations, Challenges, and Constraints

Technical limitations:

1. Market-history features are derived from the same dataset, creating a risk of subtle information leakage through area-level aggregates.

2. The dataset lacks precise coordinates (latitude/longitude), preventing spatial modeling techniques.
3. The model is trained on a static snapshot and does not account for real-time market shifts.

Resource constraints:

1. Docker environment limited to 4–8 GB RAM. A 5% sample (~79,000 rows) was used for local execution.
2. The from-scratch Gradient Boosting was limited to 10 estimators. Production implementations (XGBoost, LightGBM) with hundreds of trees would perform significantly better.
3. Extensive Neural Network hyperparameter tuning was not conducted due to time constraints.

Generalization challenges:

1. The model is trained exclusively on Dubai data and may not transfer to other markets.
2. Structural market changes (regulatory shifts, economic crises) could degrade accuracy over time.

Ethical considerations:

1. If historical data reflects systemic pricing biases, the model may perpetuate them.
2. Neural networks are less interpretable than tree-based models; stakeholders may require explainability mechanisms (e.g., SHAP values) not currently implemented.

7. Conclusions and Discussion

7.1 Summary of Findings

1. This project successfully developed an end-to-end AI pipeline for predicting residential property prices in Dubai. Key findings:
2. Feature engineering is critical: The 106-feature vector incorporating market-history aggregates, target encoding, and one-hot encoding provided a rich representation for effective modeling.
3. Bagging outperforms other Spark models: The Bagging Ensemble achieved the lowest RMSE (3,273.04 AED/sqm), showing that variance reduction through bootstrap aggregation is most beneficial for this dataset.
4. Neural Networks achieve strong explanatory power: The MLP achieved $R^2 = 0.7625$, confirming deep learning can effectively model complex real estate pricing dynamics on tabular data.
5. Gradient Boosting offers maximum consistency: Despite the highest RMSE, it had the lowest prediction variance (Std RMSE = 66.81).

7.2 Achievement of Objectives

Table5: Objective Achievement

Objective	Status
End-to-end data pipeline	Achieved
Compare ensemble methods with deep learning	Achieved
R^2 score above 0.70	Achieved
Minimize RMSE and MAE	Achieved
Reproducible Docker execution	Achieved

7.3 Key Lessons Learned

- From-scratch implementations provide deep algorithmic understanding but sacrifice the optimizations of mature libraries.
- Spark is essential for preprocessing 1.6M rows but introduces overhead for iterative algorithms where in-memory frameworks are more efficient.
- Domain knowledge drives feature engineering — features like area momentum and bedroom extraction required real estate understanding that no automated feature selection could replicate.

8. Recommendations and Future Work

For implementation:

- Deploy the Bagging Ensemble where Spark infrastructure is available for best accuracy.
- Deploy the Neural Network where R^2 and explanatory power are prioritized.
- Combine both in an ensemble meta-learner (stacking) to leverage complementary strengths.

Proposed enhancements:

- Replace from-scratch Gradient Boosting with XGBoost/LightGBM using hundreds of trees.

- Incorporate latitude/longitude for spatial modeling.
- Add time-series components (LSTM) to capture market trends.
- Use Bayesian optimization (Optuna) for systematic hyperparameter search.
- Integrate SHAP values for per-prediction explainability.

Maintenance strategy:

- Quarterly retraining with new transaction data.
- Monitor prediction error distributions to detect model drift.
- Implement schema checks on incoming data.

Broader applications:

- Adapt the pipeline for other UAE emirates.
- Extend to commercial real estate segments.
- Wrap the model in a REST API for on-demand valuation queries.

9. References

Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. In **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining** (pp. 785–794).

Friedman, J. H. (2001). *Greedy function approximation: A gradient boosting machine*. **Annals of Statistics, 29**(5), 1189–1232.

Kadra, A., Lindauer, M., Hutter, F., & Grabocka, J. (2021). *Well-tuned simple nets excel on tabular datasets*. In **Advances in Neural Information Processing Systems (NeurIPS), 34**, 23928–23941.

Kok, N., Koponen, E. L., & Martínez-Barbosa, C. A. (2014). *Big data in real estate? From manual appraisal to automated valuation*. **Journal of Portfolio Management, 43**(6), 202–211.

Limsombunchai, V. (2004). *House price prediction: Hedonic price model vs. artificial neural network*. **New Zealand Agricultural and Resource Economics Society (NZARES) Conference**.

Rosen, S. (1974). *Hedonic prices and implicit markets: Product differentiation in pure competition*. **Journal of Political Economy, 82**(1), 34–55.

Selim, H. (2009). *Determinants of house prices in Turkey: A hedonic regression model*. **Expert Systems with Applications, 36**(2), 2843–2852.

Zaharia, M., et al. (2016). *Apache Spark: A unified engine for big data processing*. **Communications of the ACM, 59**(11), 56–65.

10. Appendix

```
Project/
├── Dockerfile
├── requirements.txt
├── run_pipeline.sh
├── PricePulseAI.py              # Preprocessing (PySpark)
├── notebooks/
│   └── PricePulseAI.ipynb       # Modeling (Spark ML + PyTorch)
├── data/
│   ├── Transactions.csv         # Raw (1.6M rows, 279 MB)
│   └── Transactions_copy.csv/   # Cleaned output
├── checkpoints/                 # Spark RDD checkpoints
└── out/                         # Executed outputs
```